Secrets: what was fixed

English version of ../../security/secrets.md.

Found during the audit

Problem	Where	Risk
A Gmail app password hard-coded as a default (SMTP_PASSWORD)	server.py, now app/config.py + app/mailer.py	full access to the mailbox for anyone who saw the repository
Admin code ADMIN123 hard-coded	/api/auth/admin-login	**privilege escalation**: any signed-up user becomes an administrator
CORS allow_origins=["*"] together with allow_credentials=True and allow_origin_regex=".*"	server.py, now app/__init__.py	cross-site requests carrying session cookies
The keep-alive thread started at import time	server.py, now app/keepalive.py	network calls in tests and CI

What was done

1. SMTP credentials come from the environment only; without configuration configured is False and nothing is sent (the application still works).
2. The admin code comes from ADMIN_CODE; when it is empty the route answers **503** ("not configured") instead of letting everyone in. Comparison is secrets.compare_digest (timing-safe).
3. CORS is driven by CORS_ORIGINS, same-origin by default; methods and headers are listed explicitly.
4. Keep-alive requires KEEP_ALIVE_ENABLED=true and never starts with ENV=test.
5. Regression tests were added: tests/test_security.py fails if any of these defaults comes back.

What the owner must do

- **Revoke the Gmail app password**: it was public. Gmail → Security → App passwords → delete.
- Set ADMIN_CODE to a long random value, not ADMIN123.
- If the application was public, check whether anyone elevated their role: `SELECT id, phone, role FROM users WHERE role = 'admin';`

Rules for the future

1. No live secrets in code — only os.environ.get(...) with an empty default.
2. A missing secret disables a feature or answers 503; it never enables an unsafe mode.
3. .env is not committed (.gitignore); the example is .env.example with placeholders.
4. make doctor warns when ADMIN_CODE still looks like the example value.

History rewrite (2026-10-29)

A full history scan found a Telegram bot token in old commits (`bot/main.py`, the file was removed later) and a base64 key in `DEPLOY_RENDER.md`.

`git filter-repo --replace-text` replaced the secret literals with `REMOVED_SECRET` in all commits and all branches, and the result was force-pushed. Verification on a fresh clone: `gitleaks git` — 0 findings.

What this changes and what it does not:

1. **The Gmail app password and the bot token must still be revoked** — they were public; a history rewrite does not undo a leak.
2. `refs/pull/*` cannot be rewritten (hidden refs, force-push is rejected), so old objects stay reachable by SHA until GitHub Support removes them.
3. Clones made before the rewrite must be deleted and cloned again.